

Vishal Marandi |

BTECH/10119/21 |

Deep Learning Assignment - 3

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
```

Q1. Load the dataset and print its type and description

```
df = pd.read_csv('/content/Thyroid.txt')
print(df.dtypes)
print(df.describe())
```

on_thyroxine	int64
query_on_thyroxine	int64
on_antithyroid_medication	int64
thyroid_surgery	int64
query_hypothyroid	int64
query_hyperthyroid	int64
pregnant	int64
sick	int64
tumor	int64
lithium	int64
goitre	int64
TSH_measured	int64
T3_measured	int64
TT4_measured	int64
T4U_measured	int64
FTI_measured	int64
age	int64
sex	int64
TSH	float64
T3	float64
TT4	float64
T4U	float64
FTI	float64
classes	int64

```

dtype: object
count    on_thyroxine    query_on_thyroxine    on_antithyroid_medication \
mean      0.145305      0.017449      0.013325
std       0.352464      0.130959      0.114680
min       0.000000      0.000000      0.000000
25%      0.000000      0.000000      0.000000
50%      0.000000      0.000000      0.000000
75%      0.000000      0.000000      0.000000
max       1.000000      1.000000      1.000000

```

```

thyroid_surgery    query_hypothyroid    query_hyperthyroid
pregnant \
count    3152.000000    3152.000000    3152.000000
3152.000000
mean      0.032995      0.075825      0.077094
0.019987
std       0.178652      0.264760      0.266783
0.139979
min       0.000000      0.000000      0.000000
0.000000
25%      0.000000      0.000000      0.000000
0.000000
50%      0.000000      0.000000      0.000000
0.000000
75%      0.000000      0.000000      0.000000
0.000000
max       1.000000      1.000000      1.000000
1.000000

```

```

sick    tumor    lithium    ...    T4U_measured
FTI_measured \
count    3152.000000    3152.000000    3152.000000    ...    3152.000000
3152.000000
mean      0.030774      0.012690      0.000635    ...    0.921320
0.921637
std       0.172733      0.111952      0.025186    ...    0.269282
0.268785
min       0.000000      0.000000      0.000000    ...    0.000000
0.000000
25%      0.000000      0.000000      0.000000    ...    1.000000
1.000000
50%      0.000000      0.000000      0.000000    ...    1.000000
1.000000
75%      0.000000      0.000000      0.000000    ...    1.000000
1.000000
max       1.000000      1.000000      1.000000    ...    1.000000
1.000000

```

```

age    sex    TSH    T3    TT4

```

\	count	3152.000000	3152.000000	3152.000000	3152.000000	3152.000000
	mean	51.359772	0.295051	5.689591	1.992291	109.016783
	std	19.226176	0.456138	22.953469	1.036009	45.085350
	min	1.000000	0.000000	0.000000	0.000000	2.000000
	25%	35.000000	0.000000	0.000000	1.400000	83.000000
	50%	54.000000	0.000000	0.700000	1.900000	104.000000
	75%	67.000000	1.000000	2.200000	2.400000	128.000000
	max	98.000000	1.000000	530.000000	10.200000	450.000000

	T4U	FTI	classes
count	3152.000000	3152.000000	3152.000000
mean	0.983284	114.809613	0.091371
std	0.234424	59.028114	0.288181
min	0.000000	0.000000	0.000000
25%	0.850000	90.000000	0.000000
50%	0.960000	107.000000	0.000000
75%	1.070000	128.000000	0.000000
max	2.210000	881.000000	1.000000

[8 rows x 24 columns]

Q2. Prepare the data for Logistic Regression

```
X = df.drop('classes', axis=1)
y = df['classes']
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Logistic Regression without SGD

```
lr = LogisticRegression(max_iter=1000, C=1.0)
lr.fit(X_train, y_train)
```

```
LogisticRegression(max_iter=1000)
```

Q3. Confusion Matrix

```
y_pred = lr.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix (without SGD):\n", cm)
```

```
Confusion Matrix (without SGD):
[[552  10]
 [ 23  46]]
```

Q4. Accuracy, Precision, F-measure, Recall

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f_measure = f1_score(y_test, y_pred)
print("Without SGD - Accuracy: {}, Precision: {}, Recall: {}, F-
measure: {}".format(accuracy, precision, recall, f_measure))
```

```
Without SGD - Accuracy: 0.9477020602218701, Precision:
0.8214285714285714, Recall: 0.6666666666666666, F-measure:
0.7359999999999999
```

Q5. Logistic Regression with SGD

```
sgd = SGDClassifier(loss='log_loss', alpha=0.0001, max_iter=1000,
tol=1e-3, random_state=42)
sgd.fit(X_train, y_train)
```

```
SGDClassifier(loss='log_loss', random_state=42)
```

Q6. Confusion Matrix with SGD

```
y_pred_sgd = sgd.predict(X_test)
cm_sgd = confusion_matrix(y_test, y_pred_sgd)
print("Confusion Matrix (with SGD):\n", cm_sgd)
```

```
Confusion Matrix (with SGD):
[[562   0]
 [ 69   0]]
```

Q7. Accuracy, Precision, F-measure, Recall with SGD

```
accuracy_sgd = accuracy_score(y_test, y_pred_sgd)
precision_sgd = precision_score(y_test, y_pred_sgd)
recall_sgd = recall_score(y_test, y_pred_sgd)
f_measure_sgd = f1_score(y_test, y_pred_sgd)
print("With SGD - Accuracy: {}, Precision: {}, Recall: {}, F-measure: {}".format(accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd))
```

```
With SGD - Accuracy: 0.8906497622820919, Precision: 0.0, Recall: 0.0, F-measure: 0.0
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/metrics/_classification.py:1471: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```

Q8. Plotting the results

```
labels = ['Accuracy', 'Precision', 'Recall', 'F-measure']
without_sgd = [accuracy, precision, recall, f_measure]
with_sgd = [accuracy_sgd, precision_sgd, recall_sgd, f_measure_sgd]
x = range(len(labels))
plt.plot(x, without_sgd, label='Without SGD', marker='o')
plt.plot(x, with_sgd, label='With SGD', marker='o')
plt.xticks(x, labels)
plt.ylabel('Score')
plt.title('Comparison of Logistic Regression and SGD')
plt.legend()
plt.show()
```

Comparison of Logistic Regression and SGD

